Project Review 3: Working Prototype & Final Demonstration

Course & Project Overview

- **Course:** DATABASE SYSTEMS
- **Project Title:** Student University Management System (UMS)
- **Team Members:**
    - Jacob Jean (25BAI0077)
    - Aradhya Jain (25BCE0634)
    - Krrish Pare (25BAI0030)
- **Technology Stack:** MySQL 8.0, Python 3 (Flask), HTML5, CSS3, vanilla JavaScript
- **Source Repository:** github.com/jjean2006/University-Management-System

Review 3 turns the database designed in Review 2 into a working, role-based web application. This document covers the user interface, database connectivity, CRUD operations, search and retrieval, the key SQL queries the application runs, input validation, test cases and the final database.

1. System Overview

Architecture

The prototype follows a three-tier architecture. The browser renders HTML pages produced by Flask; Flask validates every request, runs parameterised SQL against MySQL and returns the updated page.

```
Browser (HTML5 / CSS3 / JavaScript)
        |   HTTP form posts and page requests
        v
Flask application  (app/routes/*  ->  app/validators.py  ->  app/db.py)
        |   parameterised SQL via mysql-connector / PyMySQL, transactions
        v
MySQL 8.0 database: university_db  (13 tables, 8 views)
```

Project Structure

File / Folder	Purpose
`app/__init__.py`	Flask application factory; registers one blueprint per role and the error pages
`app/config.py`	Reads database settings from the .env file (DB_USER, DB_PASSWORD, DB_HOST, DB_PORT, DB_NAME)
`app/db.py`	Connection handling, parameterised query execution, transaction helper, script runner
`app/auth.py`	Login against USER_ACCOUNT, session handling, role-based access decorator
`app/validators.py`	Server-side validation of every form field before SQL runs
`app/routes/`	admin, faculty, student, accounts, warden and auth routes
`app/templates/, app/static/`	HTML pages per role, CSS, and JavaScript (table filters, attendance sheet, payments)
`schema/schema.sql, views.sql, seed_data.sql`	DDL for 13 tables, 8 views, and the sample data
`schema/init_db.py`	Creates university_db, applies the scripts and verifies tables, row counts and views
`tests/`	306 automated tests (pytest)

User Roles

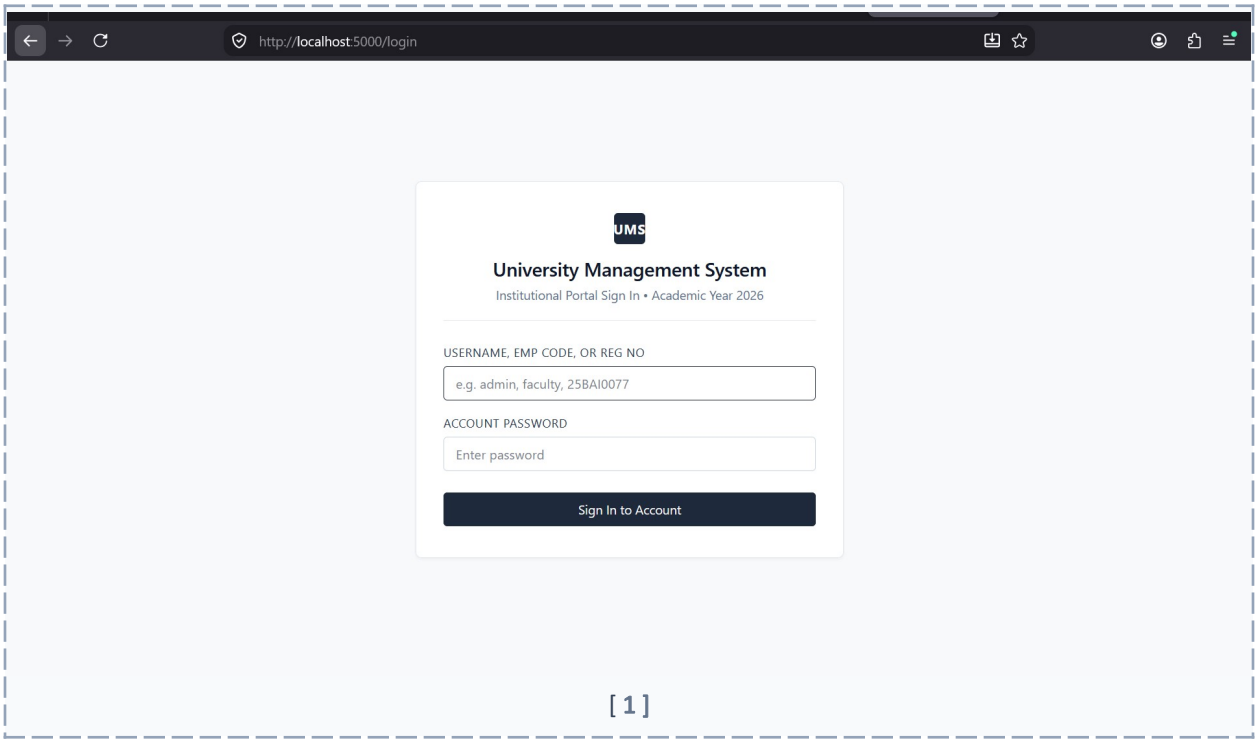
Role	Sample Login	Main Capabilities
Administrator	admin / admin123	Departments, programmes, faculty and student directories; add/edit/delete courses; add/delete students; add users; search
Faculty	EMP1001 / password	Course rosters, attendance shortage alerts (< 75%), individual and batch attendance marking
Student	25BAI0077 / password	Profile, hostel room, enrolled courses and grades, enroll / drop / waitlist, fee ledger
Accounts (Bursar)	accounts / accounts123	Issue invoices, record payments, cancel unpaid invoices, defaulters list, revenue summary
Hostel Warden	warden / warden123	Hostel blocks, room occupancy and vacancies, resident directory

2. User Interface

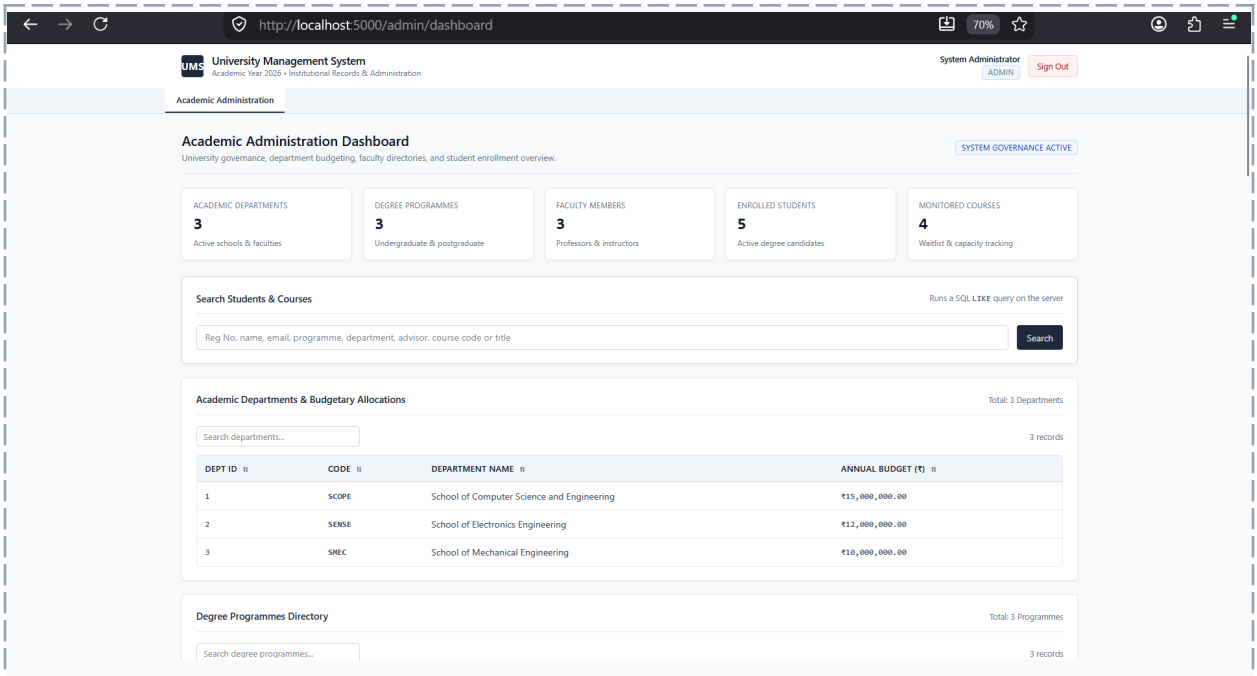
Each role gets its own dashboard built on a shared layout: a header with the signed-in user and role, a role navigation bar, and a message area that shows the success or error message after every action. Every data table has an instant filter box and sortable columns.

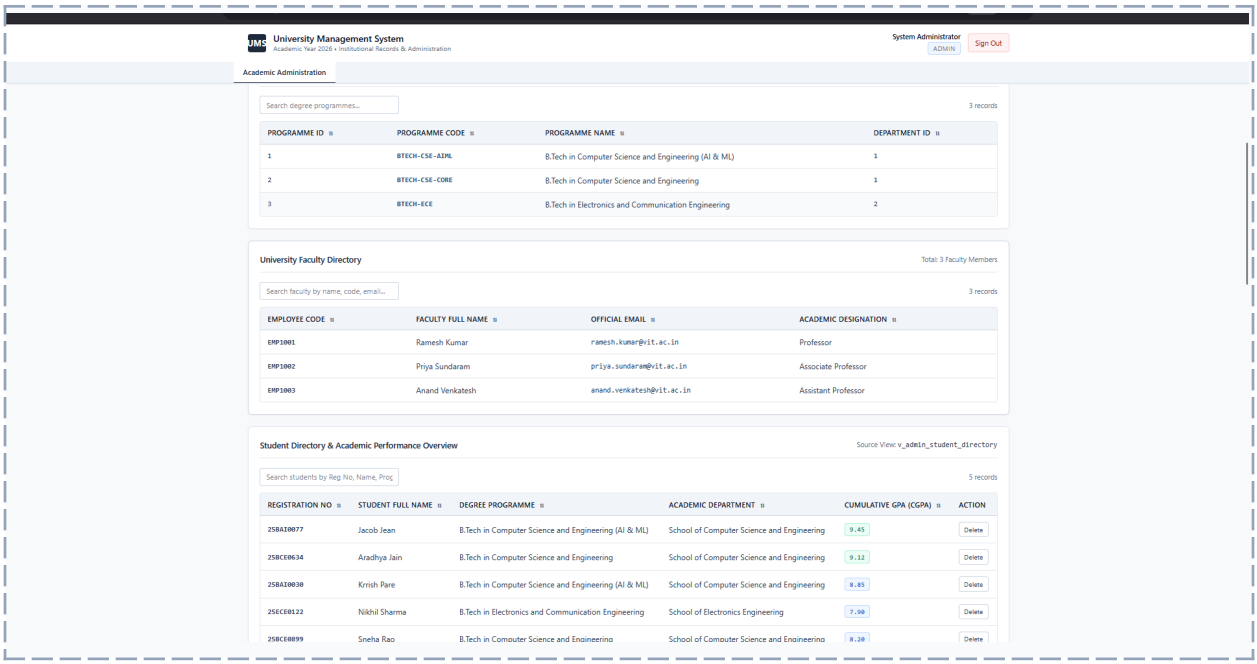
Login Page

Users sign in with their username, registration number, employee code or email. The account's role decides which dashboard opens.

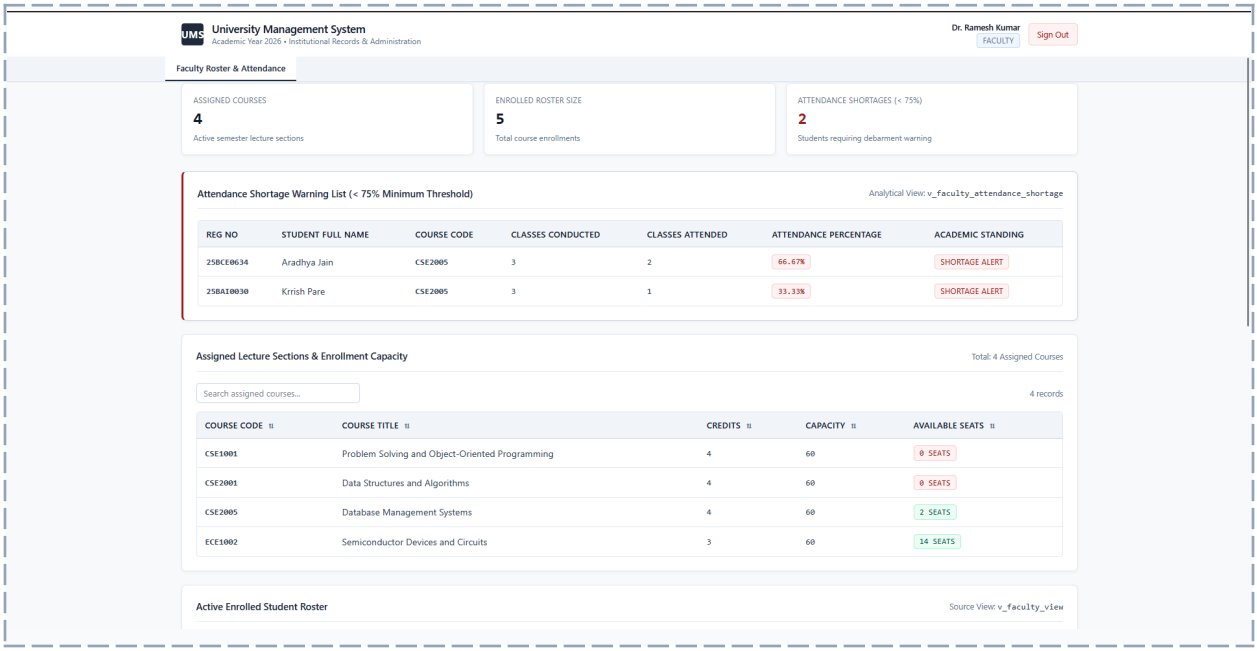


Administrator Dashboard

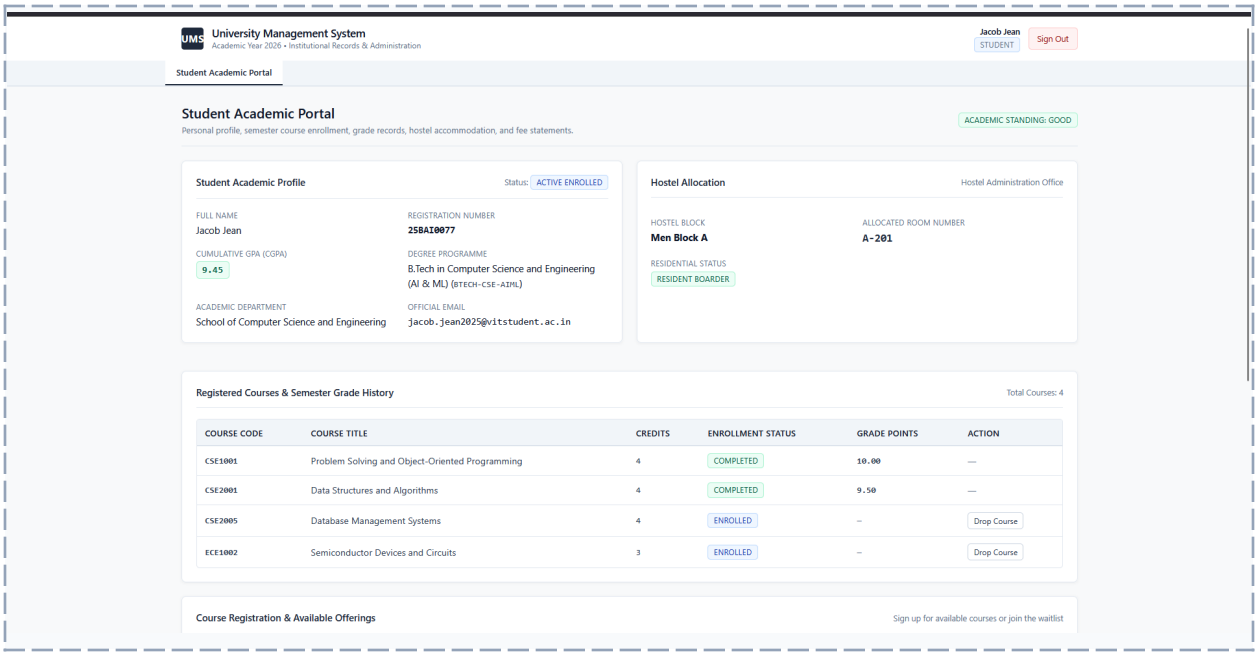




Faculty Dashboard



Student Dashboard



Course Registration & Available Offerings
Sign up for available courses or join the waitlist

Search available courses...

4 courses available

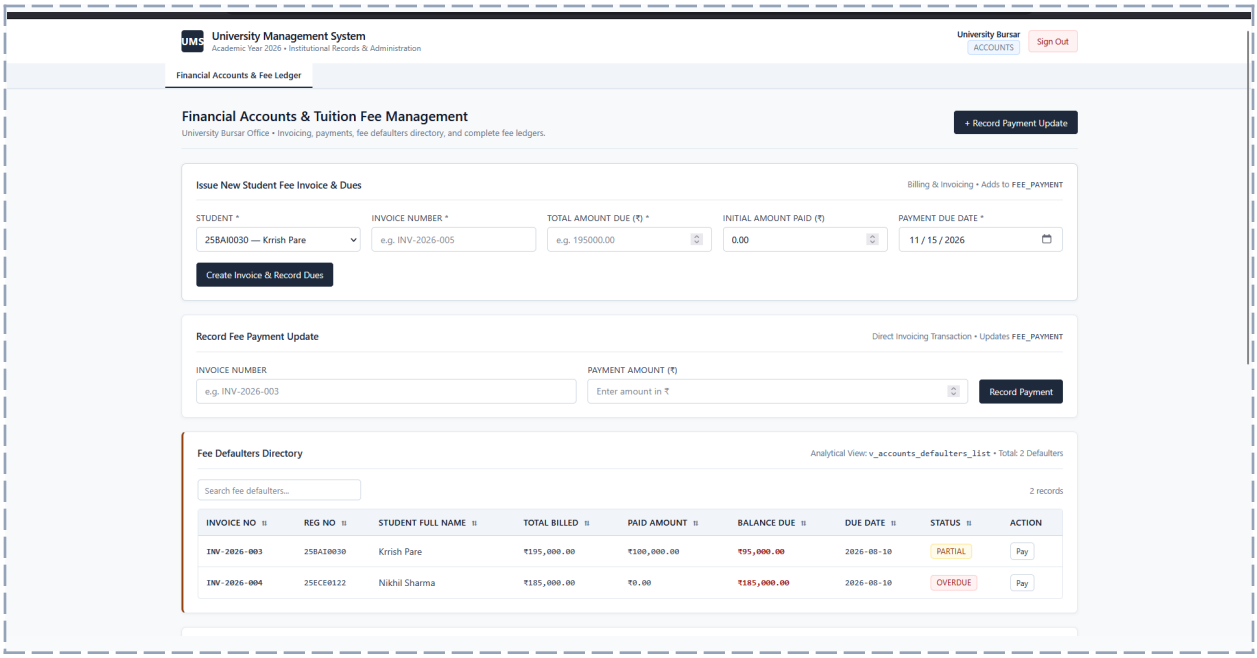
COURSE CODE	COURSE TITLE	DEPARTMENT	CREDITS	PREREQUISITE	AVAILABLE SEATS	ENROLLMENT ACTION
CSE1001	Problem Solving and Object-Oriented Programming	School of Computer Science and Engineering (SCOPE)	4	None	0 / 60	ENROLLED
CSE2001	Data Structures and Algorithms	School of Computer Science and Engineering (SCOPE)	4	CSE1001	0 / 60	ENROLLED
CSE2005	Database Management Systems	School of Computer Science and Engineering (SCOPE)	4	CSE2001	2 / 60	ENROLLED
ECE1002	Semiconductor Devices and Circuits	School of Electronics Engineering (SENSE)	3	None	14 / 60	ENROLLED

Tuition Fee Invoices & Payment Ledger

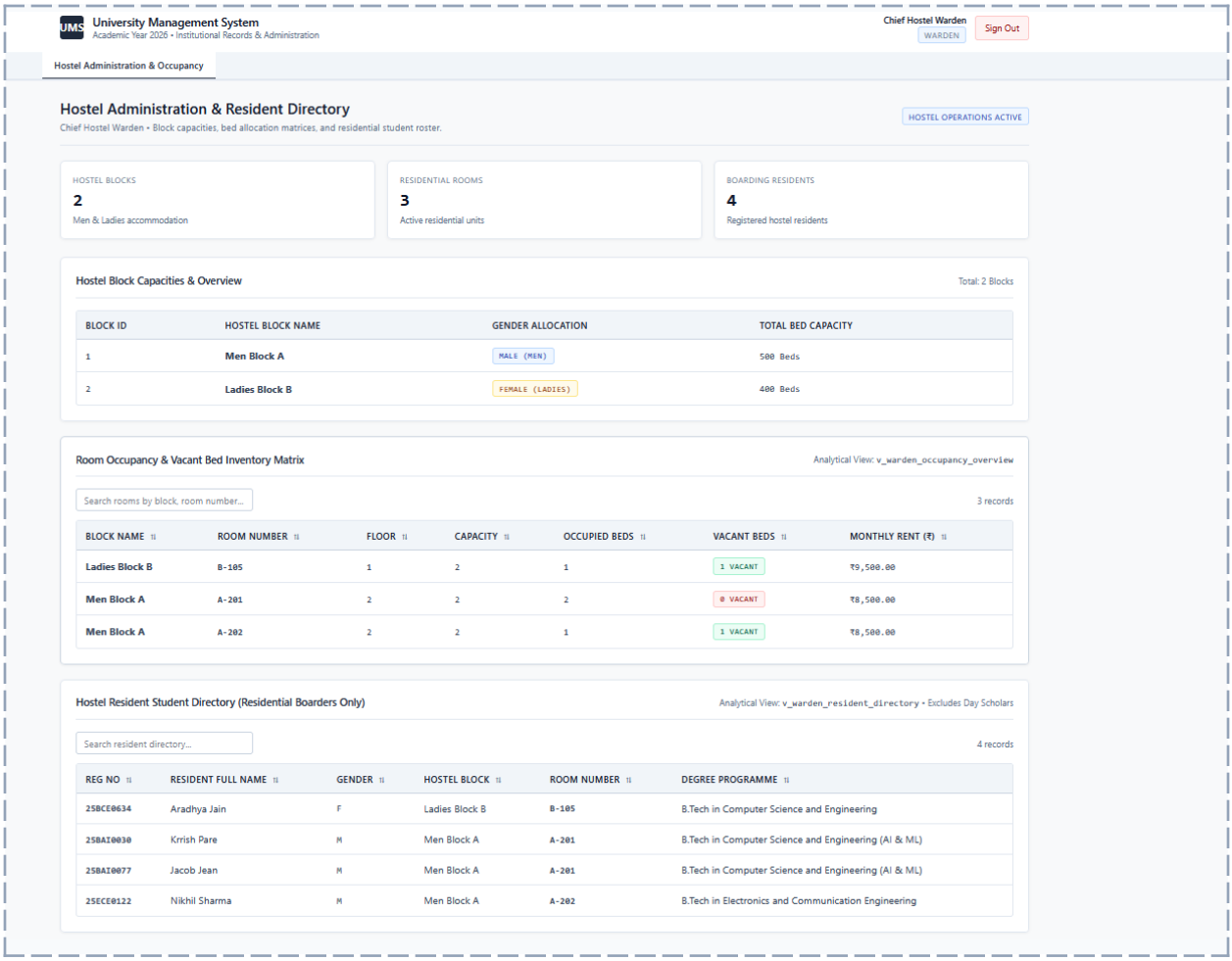
Bursar Billing Record

INVOICE NO	DESCRIPTION	TOTAL BILLED (₹)	AMOUNT PAID (₹)	DUE DATE	PAYMENT STATUS
INV-2026-001	Semester Academic Tuition Fee	₹195,000.00	₹195,000.00	2026-08-10	PAID

Accounts Dashboard



Warden Dashboard



3. Database Connectivity

Connection Configuration

Each team member keeps their own MySQL credentials in a `.env` file in the project root. The file is git-ignored, so passwords are never committed; `.env.example` is the shared template.

```
# .env  (copy of .env.example with your own values)
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=university_db
```

Connection Layer (app/db.py)

- **get_db_connection()** opens a connection with mysql-connector-python, falling back to PyMySQL and then to the mysql command-line client.
- **execute_query(sql, params)** runs one statement with %s placeholders, so user input is always sent as data and never concatenated into SQL (prevents SQL injection).
- **transaction()** runs several statements on one connection between `START TRANSACTION` and `COMMIT`, and issues `ROLLBACK` if any statement or validation check fails.
- **execute_script(sql)** runs the multi-statement schema, view and seed files (used by init_db.py) on Windows, macOS and Linux.

```
# Usage inside a route: all statements commit together or not at all
with transaction() as cur:
    cur.execute("SELECT ... FROM COURSE WHERE course_id = %s FOR UPDATE;", (course_id,))
    course = first_row(cur)
    ...
    cur.execute("INSERT INTO ENROLLMENT (...) VALUES (%s, %s, %s, CURDATE(), 'ENROLLED');",
                (...))
    cur.execute("UPDATE COURSE SET available_seats = available_seats - 1 "
                "WHERE course_id = %s;", (course_id,))
```

Database Initialisation

`python schema/init_db.py` creates `university_db`, runs schema.sql, views.sql and seed_data.sql, and then verifies every table, row count and view. Output from the final run:

```
============================================================
VERIFICATION OF TABLES & ROW COUNTS
============================================================
[+] All 13 tables exist in 'university_db'.
  - DEPARTMENT      : Expected=3, Actual=3 [OK]
  - PROGRAMME       : Expected=3, Actual=3 [OK]
  - FACULTY         : Expected=3, Actual=3 [OK]
  - SEMESTER        : Expected=2, Actual=2 [OK]
  - COURSE          : Expected=4, Actual=4 [OK]
  - HOSTEL_BLOCK    : Expected=2, Actual=2 [OK]
  - HOSTEL_ROOM     : Expected=3, Actual=3 [OK]
  - STUDENT         : Expected=5, Actual=5 [OK]
  - ENROLLMENT      : Expected=7, Actual=7 [OK]
  - ATTENDANCE      : Expected=9, Actual=9 [OK]
  - COURSE_WAITLIST : Expected=2, Actual=2 [OK]
  - FEE_PAYMENT     : Expected=4, Actual=4 [OK]
  - USER_ACCOUNT    : Expected=13, Actual=13 [OK]
[+] All table row counts match Section 4 specification exactly.

============================================================
VERIFICATION OF ANALYTICAL & OPERATIONAL VIEWS
============================================================
  - v_admin_student_directory       : Returned 5 rows.
  - v_faculty_attendance_shortage   : Returned 2 rows.
  - v_accounts_fee_summary          : Returned 3 rows.
  - v_accounts_defaulters_list      : Returned 2 rows.
  - v_warden_occupancy_overview     : Returned 3 rows.
```

```
  - v_warden_resident_directory     : Returned 4 rows.
  - v_admin_course_waitlist         : Returned 4 rows.
  - v_faculty_view                  : Returned 4 rows.

[+] SUCCESS: Database initialization and verification complete!
```

4. CRUD Operations

The table shows where each operation happens in the application. Every write goes through server-side validation first.

CRUD Matrix

Table	Create	Read	Update	Delete
COURSE	Admin: Add Course	Admin catalog, student catalog	Admin: Edit Course; seats on enroll / drop	Admin: Delete Course
STUDENT + USER_ACCOUNT	Admin: Add Student (one transaction)	Admin directory (view), student profile	Hostel bed count on allocation	Admin: Delete Student
FACULTY + USER_ACCOUNT	Admin: Add Faculty	Admin faculty directory	-	-
ENROLLMENT	Student: Enroll	Student courses, faculty roster (view)	-	Student: Drop Course
COURSE_WAITLIST	Student: Join Waitlist	Student waitlist, admin waitlist view	Re-joining reactivates the entry	Removed automatically on enrollment
ATTENDANCE	Faculty: Mark Attendance	Shortage view (< 75%)	Re-marking updates status	Removed when a course is dropped
FEE_PAYMENT	Accounts: Issue Invoice	Ledger, defaulters view, student ledger	Accounts: Record Payment	Accounts: Cancel Invoice
HOSTEL_ROOM	-	Warden occupancy view	occupied_beds on allocation / student deletion	-

Create

Add a course (admin). The code, title, credits (1-6), capacity (1-300), department and prerequisite are validated first.

```
INSERT INTO COURSE (course_code, course_title, credits, capacity, available_seats,
                    dept_id, prereq_course_id)
VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Add a student (admin). Runs as one transaction: the hostel room is locked and checked for gender and free beds, then the bed count, the STUDENT row and the login account are written together.

```
SELECT hr.room_number, hr.capacity, hr.occupied_beds, hb.gender_type, hb.block_name
FROM HOSTEL_ROOM hr JOIN HOSTEL_BLOCK hb ON hr.block_id = hb.block_id
WHERE hr.room_id = %s FOR UPDATE;

UPDATE HOSTEL_ROOM SET occupied_beds = occupied_beds + 1 WHERE room_id = %s;

INSERT INTO STUDENT (reg_no, first_name, last_name, email, dob, gender, cgpa,
                     programme_id, advisor_id, room_id)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s);

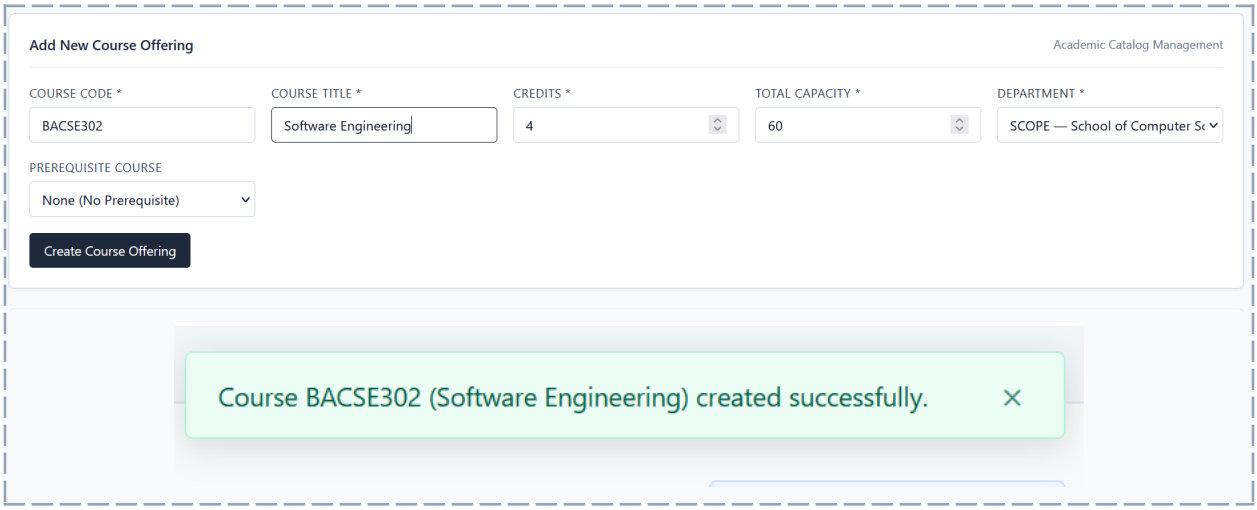
INSERT INTO USER_ACCOUNT (username, password_hash, role, entity_id, display_name, email)
VALUES (%s, %s, 'student', %s, %s, %s);
```

Issue an invoice (accounts).

```
INSERT INTO FEE_PAYMENT (student_id, invoice_no, total_amount, paid_amount, due_date,
                         payment_status, payment_date)
VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Mark attendance (faculty). Only students enrolled in the course are accepted; marking the same day again updates the status.

```
INSERT INTO ATTENDANCE (student_id, course_id, attendance_date, status)
VALUES (%s, %s, %s, %s)
ON DUPLICATE KEY UPDATE status = VALUES(status);
```



Read

Dashboards read through the Review 2 views and joins, for example:

```
SELECT * FROM v_admin_student_directory ORDER BY student_id;

SELECT c.course_id, c.course_code, c.course_title, c.credits,
       e.completion_status, e.grade_points
FROM ENROLLMENT e
JOIN COURSE c ON e.course_id = c.course_id
```

```
WHERE e.student_id = %s
ORDER BY c.course_code;
```

Update

Edit a course (admin). Capacity cannot drop below the seats already taken; available seats are recalculated.

```
SELECT course_code, capacity, available_seats FROM COURSE WHERE course_id = %s FOR UPDATE;

UPDATE COURSE SET course_title = %s, credits = %s, capacity = %s, available_seats = %s
WHERE course_id = %s;
```

Record a payment (accounts): see Section 6.2.

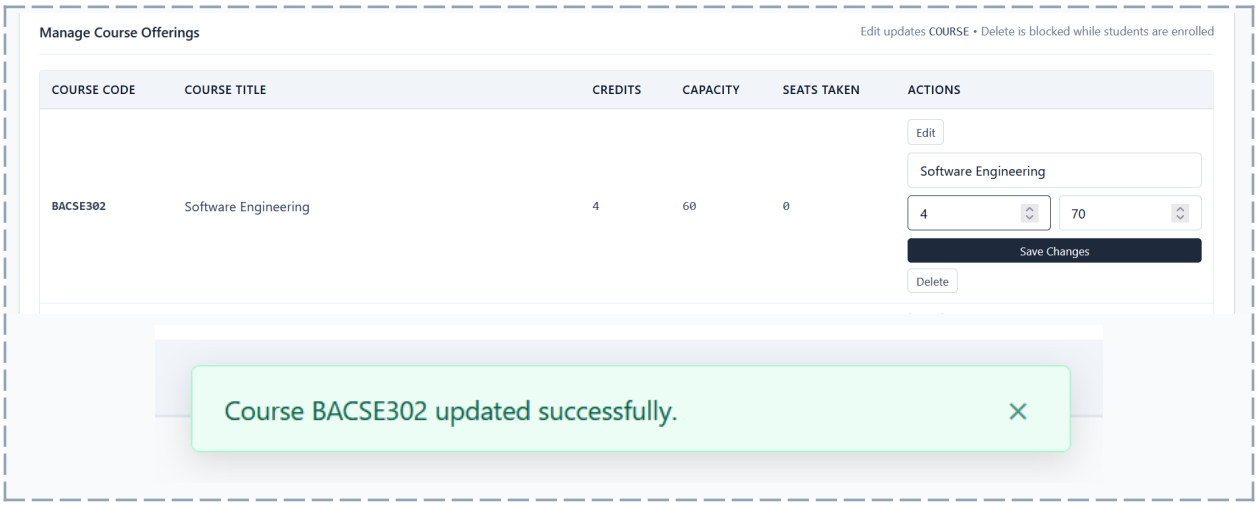


Figure S9: Admin edits a course's title, credits and capacity

Delete

Drop a course (student). Only courses with status ENROLLED can be dropped; the seat is released.

```
DELETE FROM ATTENDANCE WHERE student_id = %s AND course_id = %s;
DELETE FROM ENROLLMENT WHERE student_id = %s AND course_id = %s;
UPDATE COURSE SET available_seats = LEAST(capacity, available_seats + 1)
WHERE course_id = %s;
```

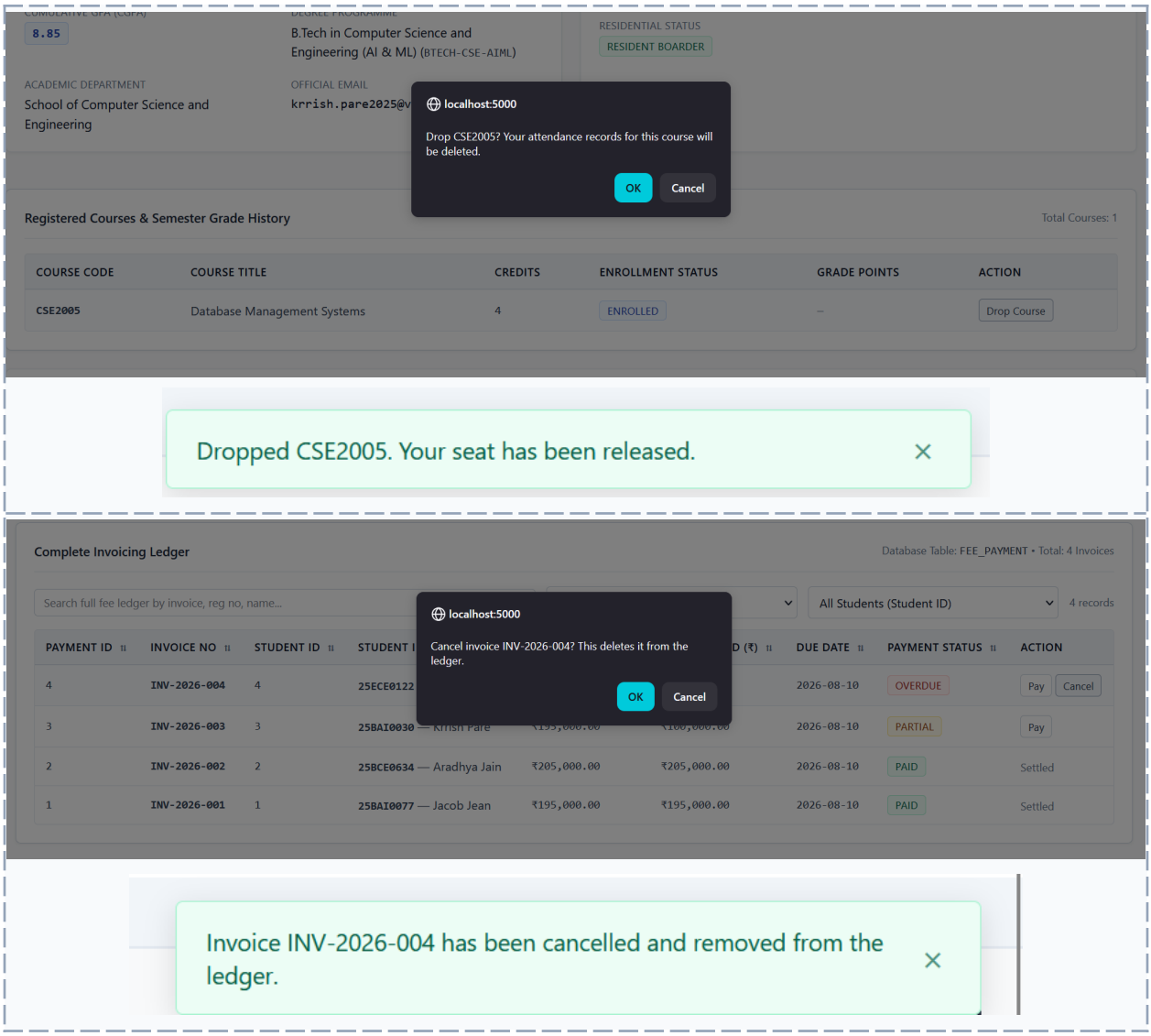
Cancel an invoice (accounts). Only invoices with no payment can be deleted.

```
DELETE FROM FEE_PAYMENT WHERE invoice_no = %s AND paid_amount = 0;
```

Delete a course (admin). Blocked while students are enrolled; otherwise ON DELETE CASCADE removes its enrollment history, attendance and waitlist rows, and ON DELETE SET NULL clears it as a prerequisite.

```
SELECT COUNT(*) AS active FROM ENROLLMENT
WHERE course_id = %s AND completion_status = 'ENROLLED';
DELETE FROM COURSE WHERE course_id = %s;
```

Delete a student (admin): see Section 6.3.



5. Search and Retrieval Operations

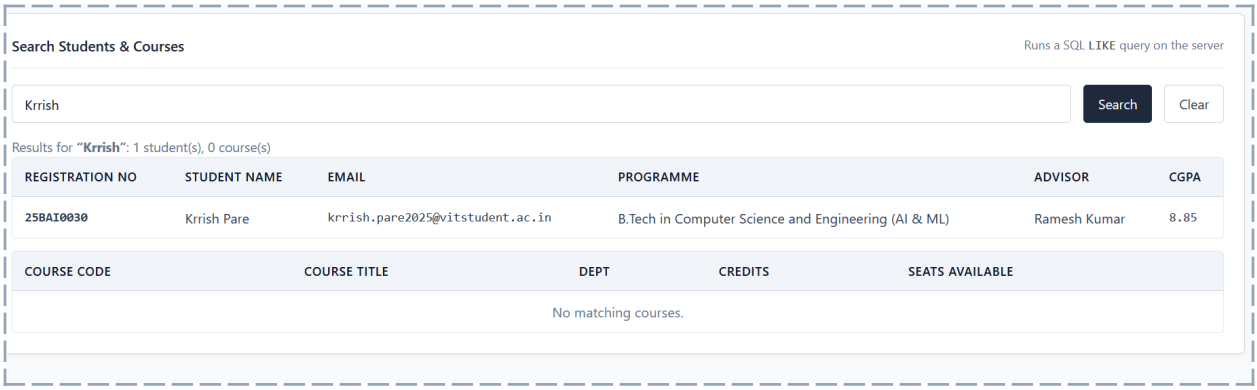
Server-side Search

The administrator dashboard has a search box that runs a SQL `LIKE` query on the server across the student directory view and the course catalog. Wildcard characters typed by the user (% and _) are escaped so they are matched literally.

```
SELECT * FROM v_admin_student_directory
WHERE reg_no LIKE %s OR student_name LIKE %s OR email LIKE %s
   OR programme_name LIKE %s OR dept_name LIKE %s OR faculty_advisor LIKE %s
ORDER BY reg_no;
```

```
SELECT c.course_code, c.course_title, c.credits, c.capacity, c.available_seats, d.dept_code
FROM COURSE c JOIN DEPARTMENT d ON c.dept_id = d.dept_id
WHERE c.course_code LIKE %s OR c.course_title LIKE %s OR d.dept_code LIKE %s
ORDER BY c.course_code;
```

Example: searching **krrish** returns 1 student (25BAI0030); searching **database** returns 1 course (CSE2005).



Retrieval Through Views

View	Used On	Retrieves
v_admin_student_directory	Admin	Students with programme, department, advisor and CGPA
v_admin_course_waitlist	Admin	Course capacity, seats and active waitlist count
v_faculty_view	Faculty	Active course rosters
v_faculty_attendance_shortage	Faculty	Students below 75% attendance per course
v_accounts_fee_summary	Accounts	Billed, collected and outstanding totals per status
v_accounts_defaulters_list	Accounts	Invoices that are PENDING, PARTIAL or OVERDUE
v_warden_occupancy_overview	Warden	Room capacity, occupied and vacant beds
v_warden_resident_directory	Warden	Residents by block and room

Client-side Filters

Every dashboard table also has an instant filter box and sortable column headers, and the accounts ledger can be filtered by payment status and student.

6. Important SQL Queries Through the Application

6.1 Course Enrollment (transaction with row lock)

The course row is locked with `FOR UPDATE` so two students cannot take the last seat at the same time. The prerequisite is checked, then the enrollment, seat update and waitlist cleanup commit together.

```
START TRANSACTION;
SELECT c.course_id, c.course_code, c.course_title, c.available_seats, c.prereq_course_id,
       p.course_code AS prereq_code
FROM COURSE c LEFT JOIN COURSE p ON c.prereq_course_id = p.course_id
WHERE c.course_id = %s FOR UPDATE;

SELECT 1 AS ok FROM ENROLLMENT
WHERE student_id = %s AND course_id = %s
  AND completion_status = 'COMPLETED';                        -- prerequisite check

SELECT semester_id FROM SEMESTER WHERE is_active = 1 LIMIT 1;

INSERT INTO ENROLLMENT (student_id, course_id, semester_id, enrollment_date,
                        completion_status)
VALUES (%s, %s, %s, CURDATE(), 'ENROLLED');
UPDATE COURSE SET available_seats = available_seats - 1 WHERE course_id = %s;
DELETE FROM COURSE_WAITLIST WHERE student_id = %s AND course_id = %s;
COMMIT;
```

6.2 Recording a Fee Payment

The invoice is locked; payments above the outstanding balance are rejected. The new status is PAID when the balance reaches zero, OVERDUE if the due date has passed, otherwise PARTIAL.

```
START TRANSACTION;
SELECT total_amount, paid_amount, due_date, payment_status
FROM FEE_PAYMENT WHERE invoice_no = %s FOR UPDATE;

UPDATE FEE_PAYMENT SET paid_amount = %s, payment_status = %s, payment_date = CURDATE()
WHERE invoice_no = %s;
COMMIT;
```

6.3 Deleting a Student

Seats in the student's active courses and their hostel bed are released first; deleting the STUDENT row then cascades to ENROLLMENT, ATTENDANCE, COURSE_WAITLIST and FEE_PAYMENT through ON DELETE CASCADE.

```
START TRANSACTION;
```

```
UPDATE COURSE c JOIN ENROLLMENT e ON e.course_id = c.course_id
SET c.available_seats = LEAST(c.capacity, c.available_seats + 1)
WHERE e.student_id = %s AND e.completion_status = 'ENROLLED';

UPDATE HOSTEL_ROOM SET occupied_beds = GREATEST(occupied_beds - 1, 0) WHERE room_id = %s;
DELETE FROM USER_ACCOUNT WHERE role = 'student' AND entity_id = %s;
DELETE FROM STUDENT WHERE student_id = %s;
COMMIT;
```

6.4 Attendance Shortage (view with GROUP BY and HAVING)

```
CREATE VIEW v_faculty_attendance_shortage AS
SELECT c.course_code, c.course_title, s.reg_no,
       CONCAT(s.first_name, ' ', s.last_name) AS student_name, s.email AS student_email,
       COUNT(a.attendance_date) AS total_classes_conducted,
       SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END) AS classes_attended,
       ROUND((SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END)
              / COUNT(a.attendance_date)) * 100, 2) AS attendance_percentage
FROM ATTENDANCE a
JOIN COURSE c ON a.course_id = c.course_id
JOIN STUDENT s ON a.student_id = s.student_id
GROUP BY c.course_id, c.course_code, c.course_title,
         s.student_id, s.reg_no, s.first_name, s.last_name
HAVING attendance_percentage < 75.00;
```

6.5 Login

The account is found by username or email and the password must match that account exactly.

```
SELECT user_id, username, password_hash, role, entity_id, display_name, email
FROM USER_ACCOUNT
WHERE UPPER(username) = UPPER(%s) OR UPPER(email) = UPPER(%s)
ORDER BY UPPER(username) = UPPER(%s) DESC, user_id;
```

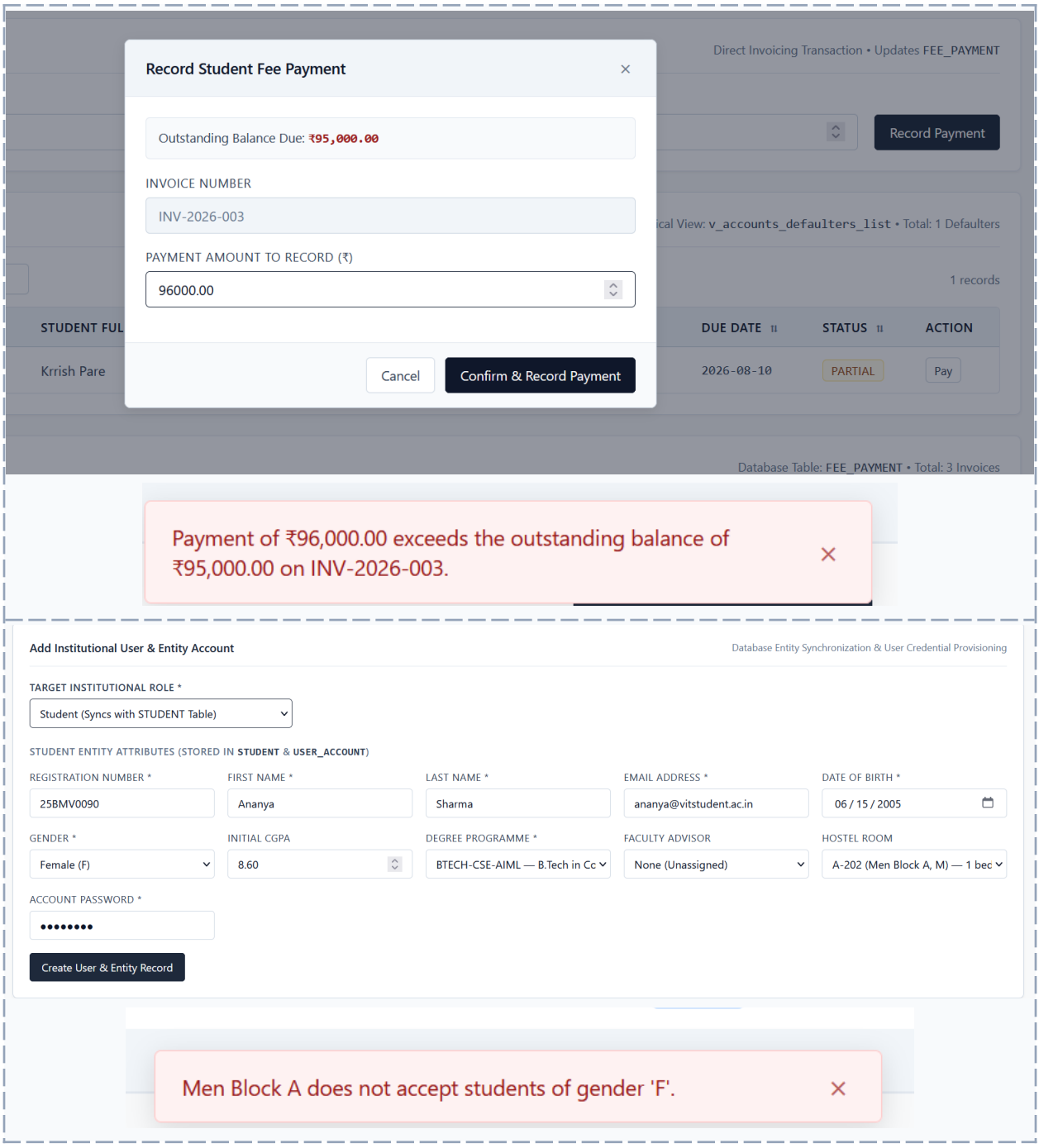
7. Input Validation

Input is checked in three layers:

- **Browser:** HTML form attributes (`required`, `type=email`, `type=date`, `min`, `max`) catch simple mistakes before submitting.
- **Server:** `app/validators.py` checks every field again in Flask before any SQL runs, and the route checks business rules (seats, balances, room gender, enrollment). Errors are shown as a red message on the page.
- **Database:** PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and DECIMAL precision constraints from Review 2 reject anything that gets past the first two layers.

Form	Rule	Message Shown When the Rule Fails
Login	Password must match that account exactly	Invalid credentials. Please verify your username and password.

Form	Rule	Message Shown When the Rule Fails
Add Student	Reg No format NNXXXNNNN, valid email, letters-only names	Registration number 'BAD' is not valid (expected format like 25BCE1001).
Add Student	CGPA between 0.00 and 10.00	CGPA must be between 0.00 and 10.00.
Add Student	Hostel block must match the student's gender	Men Block A does not accept students of gender 'F'.
Add Student	Room must have a free bed	Room A-201 is full.
Add User	Username must not already exist	A login account with username '25BAI0030' already exists.
Add Course	Credits 1-6, capacity 1-300, code like CSE3001	Credits must be at least 1.
Edit Course	Capacity not below seats taken	Capacity cannot be reduced below the 60 seats already taken in CSE1001.
Delete Course	No students currently enrolled	CSE2005 has 3 student(s) currently enrolled and cannot be deleted.
Issue Invoice	Paid amount not negative	Initial amount paid cannot be less than ₹0.00.
Issue Invoice	Paid amount not above total	Initial amount paid cannot be more than the total amount.
Record Payment	Not above outstanding balance	Payment of ₹95,000.01 exceeds the outstanding balance of ₹95,000.00 on INV-2026-003.
Record Payment	Invoice not already paid	Invoice INV-2026-001 is already fully paid.
Cancel Invoice	Only invoices with no payments	Invoice INV-2026-004 already has payments recorded and cannot be cancelled.
Attendance	Student must be enrolled in the course	None of the selected students are enrolled in CSE2005.
Attendance	Date not in the future	Lecture date cannot be after <today's date>. (e.g. 2026-10-19 on the test run)
Attendance	Status must be Present or Absent	Attendance status must be one of: Present, Absent.
Join Waitlist	Only when the course is full	CSE2005 still has open seats — enroll directly instead of joining the waitlist.
Enroll	Prerequisite must be completed	Prerequisite course CSE2001 must be completed before enrolling in CSE2005.
Drop Course	Completed courses cannot be dropped	CSE1001 is already COMPLETED and cannot be dropped.



8. Test Cases

Test File	What It Tests	Tests	Result
`test_tier1_features.py`	Connectivity, tables, views, seed data, login and every dashboard	65	65 passed

Test File	What It Tests	Tests	Result
`test_tier2_boundaries.py`	Boundary inputs, invalid data, constraints and role restrictions	73	73 passed
`test_tier3_pairwise.py`	Actions reflected across dashboards (fees, attendance, waitlists, hostel)	17	17 passed
`test_tier4_scenarios.py`	End-to-end real-world scenarios	6	6 passed
`test_tier5_crud_validation.py`	CRUD operations, search and input validation added for Review 3	40	40 passed
`test_challenger_m1_empirical.py`	FK RESTRICT / CASCADE / SET NULL, UNIQUE and composite keys, connections	22	20 passed, 2 skipped
`test_challenger_m2_empirical.py`	Authentication, sessions, cross-role access, security regressions	83	83 passed
`Total`		306	304 passed, 2 skipped

Functional Test Cases

Representative test cases executed against the running application (actual results are the messages and database changes observed).

ID	Test Case	Input	Expected Result	Actual Result	Status
TC01	Valid login	admin / admin123	Admin dashboard opens	Redirected to /admin/dashboard	Pass
TC02	Wrong password for account	admin / student123	Login rejected	HTTP 401, Invalid credentials	Pass
TC03	Role restriction	Student opens /admin/dashboard	Access denied	HTTP 403 Forbidden	Pass
TC04	Not logged in	Open /accounts/dashboard	Sent to login	Redirected to /login	Pass
TC05	Enroll in course	25BCE0899 enrolls in ECE1002	Enrollment row added, seats 15 → 14	Enrolled; available_seats = 14	Pass
TC06	Drop course	25BCE0899 drops ECE1002	Row deleted, seat released	Dropped; available_seats = 15	Pass
TC07	Drop completed course	25BAI0077 drops CSE1001	Rejected	CSE1001 is already COMPLETED and cannot be dropped.	Pass
TC08	Waitlist with open seats	25BCE0899 joins CSE2005 waitlist	Rejected	CSE2005 still has open seats …	Pass
TC09	Issue invoice	INV-T-3, ₹1,000, paid 0	Invoice created as PENDING	Issued; status PENDING	Pass
TC10	Negative paid amount	Paid = -500	Rejected	Initial amount paid cannot be less than ₹0.00.	Pass

ID	Test Case	Input	Expected Result	Actual Result	Status
TC11	Overpayment	INV-2026-003, ₹95,000.01	Rejected	Payment … exceeds the outstanding balance …	Pass
TC12	Exact balance payment	INV-2026-003, ₹95,000	Status becomes PAID	Status updated to PAID	Pass
TC13	Part-payment on overdue invoice	INV-2026-004, ₹1,000	Stays OVERDUE	Status updated to OVERDUE	Pass
TC14	Cancel unpaid invoice	Cancel INV-T-3	Row deleted	Invoice INV-T-3 has been cancelled …	Pass
TC15	Attendance for non-enrolled student	Course CSE2005, student 25BCE0899	Rejected	None of the selected students are enrolled in CSE2005.	Pass
TC16	Future attendance date	Date 2030-01-01	Rejected	Lecture date cannot be after 2026-10-19.	Pass
TC17	Batch attendance	3 students, 1 not enrolled	2 recorded, 1 skipped	Recorded 2 … 1 student(s) skipped	Pass
TC18	Add course with invalid values	Credits -4, capacity -10	Rejected	Credits must be at least 1.	Pass
TC19	Add, edit and delete course	CSE3001 → edit → delete	Row created, updated, removed	Created; updated to 3 credits / 50 seats; deleted	Pass
TC20	Delete course with enrollments	Delete CSE2005	Rejected	CSE2005 has 3 student(s) currently enrolled …	Pass
TC21	Add student to full room	Room A-201 (2/2 beds)	Rejected	Room A-201 is full.	Pass
TC22	Add then delete student	25BCE9998 in room B-105	Beds 1 → 2 → 1; login removed	Added, can log in; deleted, bed released	Pass
TC23	Overwrite existing account	New warden named 25BAI0030	Rejected	A login account with username '25BAI0030' already exists.	Pass
TC24	Server-side search	q = krrish	Matching student listed	1 student(s): 25BAI0030	Pass
TC25	Wildcard in search	q = %	Treated literally	0 student(s), 0 course(s)	Pass

9. Final Database

Database Objects

The final database `university_db` contains 13 tables and 8 views (from the init_db.py verification above).

Table	Purpose	Sample Rows
DEPARTMENT	Academic schools and budgets	3
PROGRAMME	Degree programmes per department	3
FACULTY	Faculty members	3

Table	Purpose	Sample Rows
SEMESTER	Academic terms	2
COURSE	Course catalog with capacity and prerequisite	4
HOSTEL_BLOCK	Hostel buildings	2
HOSTEL_ROOM	Rooms, capacity and occupied beds	3
STUDENT	Student records	5
ENROLLMENT	Student-course registrations and grades	7
ATTENDANCE	Daily lecture attendance	9
COURSE_WAITLIST	Waitlist queue for full courses	2
FEE_PAYMENT	Invoices and payments	4
USER_ACCOUNT	Login accounts and roles (new in Review 3)	13

10. Demonstration Flow

Suggested order for the live demo (start from a fresh database with `python schema\init_db.py`, then `python run.py`):

- **Step 1 – Connectivity:** show `.env`, run init_db.py, and show the verification output.
- **Step 2 – Admin:** log in as admin, search for a student, add course CSE3001, edit it, then delete it; try deleting CSE2005 to show the block.
- **Step 3 – Admin:** add a student with an invalid room to show validation, then add them correctly and log in as that student.
- **Step 4 – Student:** log in as 25BCE0899, enroll in ECE1002 (seat count drops), then drop it (seat count returns).
- **Step 5 – Faculty:** log in as EMP1001, pick CSE2005 in the batch sheet (only enrolled students appear) and submit attendance; show the shortage list.
- **Step 6 – Accounts:** issue an invoice, record a payment, try an overpayment to show validation, then cancel an unpaid invoice.
- **Step 7 – Warden:** show room occupancy and the resident directory.

11. Limitations and Future Enhancements

- **Course–faculty assignment:** the schema has no table linking courses to the faculty who teach them, so every faculty member sees all courses. A COURSE_FACULTY table would restrict each faculty member to their own sections.
- **Password storage:** passwords are stored as plain text for this academic prototype; a production system would store salted hashes.
- **Single active semester:** enrollment always uses the active semester; managing semesters from the UI is a future enhancement.
- **Student and faculty edits:** profile details are created and deleted by the administrator but not yet edited from the UI.

12. Conclusion

The University Management System is a working prototype connected to the MySQL database designed in Reviews 1 and 2. It provides a simple role-based interface, full create, read, update and delete operations, server-side search, validated input, transactional updates for multi-step changes.

PROJECT REVIEW 3 · DATABASE SYSTEMS

Working Prototype & Final Demonstration

Student University Management System (UMS)

Jacob Jean
25BAI0077

Aradhya Jain
25BCE0634

Krrish Pare
25BAI0030

Review 3 at a Glance

The Review 2 database is now a working web application with a role-based interface connected to MySQL.

5

role-based dashboards

13 + 8

tables + views in MySQL

20

validation rules enforced

304

of 306 automated tests passing



Three-Tier Architecture

Every request is validated in Flask, then runs as parameterised SQL against MySQL.



Browser

HTML5 · CSS3 · JavaScript

One dashboard per role, instant table filters, live attendance sheet





Flask Application

Python 3 · Flask

Role-based routes, server-side validation, transactions





MySQL 8.0

university_db

13 tables, 8 views, keys and constraints from Review 2

`HTTP form posts → routes → validators.py → db.py → mysql-connector / PyMySQL`

Simple, Role-Based Interface

Each user signs in once and lands on the dashboard for their role.



Admin

`admin / admin123`

Directories, courses, users, search



Faculty

`EMP1001 / password`

Rosters, attendance, shortage alerts



Student

`25BAI0077 / password`

Profile, enroll, drop, waitlist, fees



Accounts

`accounts / accounts123`

Invoices, payments, defaulters



Warden

`warden / warden123`

Rooms, vacancies, residents

Shared layout: signed-in user and role in the header, a success or error message after every action, filter and sort on every table.

Database Connectivity

Each member keeps their own MySQL login in a git-ignored .env file; the code is identical for everyone.

```
# .env  (from .env.example)
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=university_db
```

```
> python schema\init_db.py
[+] All 13 tables exist in 'university_db'.
[+] All table row counts match Section 4.
[+] SUCCESS: initialization complete!
```

Parameterised queries

Every value is sent with %s placeholders, so input can never change the SQL (no SQL injection).

Transactions

Multi-step changes run between START TRANSACTION and COMMIT, with ROLLBACK on any error.

One-command setup

init_db.py creates the database, runs schema, views and seed data, then verifies them.

CRUD Operations

Create, read, update and delete are all available from the interface.

Table	Create	Read	Update	Delete
`COURSE`	Admin: Add Course	Admin & student catalogs	Admin: Edit Course	Admin: Delete Course
`STUDENT + login`	Admin: Add Student	Directory view, profile	Bed count on allocation	Admin: Delete Student
`ENROLLMENT`	Student: Enroll	Courses, faculty roster	Seats on enroll / drop	Student: Drop Course
`ATTENDANCE`	Faculty: Mark	Shortage view (< 75%)	Re-marking a day	Removed on drop
`FEE_PAYMENT`	Accounts: Issue Invoice	Ledger, defaulters view	Accounts: Record Payment	Accounts: Cancel Invoice
`COURSE_WAITLIST`	Student: Join	Student & admin views	Re-join reactivates	Cleared on enrollment

Key SQL: Enrolling in a Course

One transaction with a row lock, so two students can never take the last seat.

```
START TRANSACTION;
SELECT course_code, available_seats,
       prereq_course_id
FROM COURSE
WHERE course_id = %s FOR UPDATE;

INSERT INTO ENROLLMENT (student_id, course_id,
  semester_id, enrollment_date, completion_status)
VALUES (%s, %s, %s, CURDATE(), 'ENROLLED');
UPDATE COURSE SET available_seats =
  available_seats - 1 WHERE course_id = %s;
DELETE FROM COURSE_WAITLIST
WHERE student_id = %s AND course_id = %s;
COMMIT;
```

1. Lock the course row
2. Check prerequisite and free seats
3. Insert enrollment, take a seat
4. Clear any waitlist entry

Key SQL: Deleting a Student

Seats and the hostel bed are released first; foreign keys then cascade to every dependent table.

```
START TRANSACTION;
UPDATE COURSE c JOIN ENROLLMENT e
  ON e.course_id = c.course_id
SET c.available_seats =
  LEAST(c.capacity, c.available_seats + 1)
WHERE e.student_id = %s
  AND e.completion_status = 'ENROLLED';
UPDATE HOSTEL_ROOM SET occupied_beds =
  GREATEST(occupied_beds - 1, 0) WHERE room_id = %s;
DELETE FROM USER_ACCOUNT
WHERE role = 'student' AND entity_id = %s;
DELETE FROM STUDENT WHERE student_id = %s;
COMMIT;
```



DELETE FROM STUDENT

→ ON DELETE CASCADE → ENROLLMENT

→ ON DELETE CASCADE → ATTENDANCE

→ ON DELETE CASCADE → COURSE_WAITLIST

→ ON DELETE CASCADE → FEE_PAYMENT

Also: delete course (blocked while students are enrolled), drop course, cancel unpaid invoice.

Search and Retrieval

The admin search runs a SQL LIKE query on the server; dashboards read through the Review 2 views.

```
SELECT * FROM v_admin_student_directory
WHERE reg_no LIKE %s
   OR student_name LIKE %s
   OR email LIKE %s
   OR programme_name LIKE %s
   OR dept_name LIKE %s
   OR faculty_advisor LIKE %s
ORDER BY reg_no;
```



“krrish” → 25BAI0030 Krrish Pare

% and _ are escaped, so typed wildcards match literally

8 views in use

- v_admin_student_directory
- v_admin_course_waitlist
- v_faculty_view
- v_faculty_attendance_shortage
- v_accounts_fee_summary
- v_accounts_defaulters_list
- v_warden_occupancy_overview
- v_warden_resident_directory

Input Validation in Three Layers

Bad input is stopped before it reaches the data, and the user is told exactly why.



Browser

required, type=email / date, min and max on every form field



Flask server

validators.py re-checks every field, then business rules: seats, balances, room gender, enrollment



MySQL

PRIMARY / FOREIGN / UNIQUE keys, NOT NULL and DECIMAL precision from Review 2

Real messages from the app

- Room A-201 is full.
- Men Block A does not accept students of gender 'F'.
- Payment of ₹95,000.01 exceeds the outstanding balance of ₹95,000.00 on INV-2026-003.
- Initial amount paid cannot be more than the total amount.
- CSE2005 has 3 student(s) currently enrolled and cannot be deleted.

20 rules in total – full list in the report, Section 7.

Final Database and Changes Since Review 2

Two changes since Review 2; USER_ACCOUNT is also added to the ER diagram to keep it consistent.

university_db

DEPARTMENT
PROGRAMME
FACULTY
SEMESTER
COURSE
HOSTEL_BLOCK
HOSTEL_ROOM
STUDENT
ENROLLMENT
ATTENDANCE
COURSE_WAITLIST
FEE_PAYMENT
USER_ACCOUNT

13 tables · 8 views · 47 records + 13 logins

New: USER_ACCOUNT

Logins for all five roles; entity_id links an account to its STUDENT or FACULTY row.

Fix: grade_points DECIMAL(4,2)

DECIMAL(3,2) stops at 9.99, so a grade of 10.00 could not be stored.

Known limitation

No course–faculty table yet, so every faculty member sees all courses.

Live Demonstration Flow

Fresh database, then one pass through every role.

1 Setup

Run init_db.py, start run.py

2 Admin

Search, add / edit / delete course, add student

3 Student

Enroll in ECE1002, then drop it

4 Faculty

Batch attendance for CSE2005

5 Accounts

Invoice, payment, overpayment error

6 Warden

Occupancy and residents